

Error Handling in the Reactive World

In reactive programming, streams are **asynchronous** and **non-blocking**. Code execution continues immediately after calling `subscribe()`, so traditional `try/catch` blocks cannot catch errors that happen later in the pipeline.

Reactive Streams use a special error channel: `onError` event. You must handle errors **declaratively** using dedicated operators.

1. `onErrorComplete()` — Catch and swallow the error

If you don't even want to replace the exception with a fallback value, but instead to ignore it and only propagate elements that have been produced so far, what you want is essentially replacing the `onError` signal with an `onComplete` signal.

```
Flux.just("apple", "banana", "cherry")
    .map(fruit -> {
        if (fruit.equals("banana")) throw new RuntimeException("Bad fruit!");
        return fruit.toUpperCase();
    })
    .onErrorComplete()
    .subscribe(System.out::println);
```

2. `onErrorReturn()` – Catching and Replacing Errors

Returns a **default value** when an error occurs and then completes the stream.

```
Flux.just("apple", "banana", "cherry")
    .map(fruit -> {
        if (fruit.equals("banana")) throw new RuntimeException("Bad fruit!");
        return fruit.toUpperCase();
    })
```

```
.onErrorReturn("DEFAULT_FRUIT")  
.subscribe(System.out::println);
```

Output:

```
APPLE  
DEFAULT_FRUIT
```

3. `onErrorResume()` – Falling Back to Another Stream

When an error occurs, **switch** to a completely different stream (alternative Flux/Mono).

```
Flux.just("apple", "banana", "cherry")  
    .map(fruit -> {  
        if (fruit.equals("banana")) throw new RuntimeException("Bad fruit!");  
        return fruit.toUpperCase();  
    })  
    .onErrorResume(err -> Flux.just("mango", "orange")) // fallback stream  
    .subscribe(System.out::println);
```

Output:

```
APPLE  
mango  
orange
```

Use Case: Call another service, read from cache, or return backup data when the main source fails.

4. `onErrorMap()` – Transforming Exceptions

Changes one exception type into another (useful for wrapping or standardizing errors).

```

Flux.just("apple", "banana", "cherry")
    .map(fruit -> {
        if (fruit.equals("banana")) throw new RuntimeException("Bad fruit!");
        return fruit.toUpperCase();
    })
    .onErrorMap(err -> new BusinessException("Failed to process fruits", err))
    .subscribe(
        System.out::println,
        err -> System.err.println("Error: " + err.getMessage())
    );

```

Use Case: Convert low-level exceptions (e.g., `WebClientException`) into domain-specific exceptions.

Global Error Handling

You can also use `doOnError()` for side effects (like logging) without recovering:

```

...
.doOnError(err -> System.err.println("Logged error: " + err))

```